Security

NestUp

Authentication, authorisation, secrets, and remaining risks

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

Posture: 21 tables, **RLS enabled on every one**, 72 policies · authorisation enforced in Postgres, not in application code

---

1. The governing principle

Almost every decision in this document follows from one:

> **The application is not trusted to decide who may see what. Postgres is.**

Row Level Security means the database itself refuses a query for rows the caller may not read. A bug in a page, a Server Action or a cache cannot leak another member's data, because the leak would have to get past the database first — and the database does not know or care which line of TypeScript asked.

This matters more here than in most products, because NestUp holds phone numbers, home addresses, private messages and the times people have arranged to meet strangers.

---

2. Authentication

Supabase Auth, email + password.

- **Email confirmation is mandatory.** A deliberate decision, kept against the temptation to remove it during development. In a product where people meet in real life, it is the cheapest barrier that exists against throwaway accounts. Signing up creates the row and mails a 6-digit code; the session is **withheld until the code is entered** — `tests/unit/signup-confirmation.test.ts` exists specifically to stop that regressing.
- **Passwords are never handled by us.** Supabase Auth hashes and stores them; the application never sees, logs or stores a password.
- **Sessions are httpOnly cookies**, managed by `@supabase/ssr`, refreshed by the edge proxy on every request.
- **Password recovery** via emailed link → `/auth/confirm?type=recovery` → `/reset-password`.
- **Changing an email or password requires re-authentication** with the current password.
- **Auth email is rate-limited** by the `auth_mail_throttle` table, so the endpoint cannot be used to spam an address.

Sign-out is a security operation, and is treated as one

Signing out posts to `/auth/signout`, a Route Handler that ends the session and answers **HTTP 303**.

The 303 is the point. A Server Action's `redirect()` is a *soft* navigation: the React client survives it, and so does everything in memory — including this browser's `"use cache: private"` entries (deck, inbox, profile tabs) and the router's cache of already-rendered pages. The member saw a signed-out page with their own data still sitting behind it, one back-button press away, and **the next person to sign in on that computer inherited the tab in that state.**

A 303 forces a real document navigation, which tears the client down and rebuilds it empty. Nothing in `next/cache` can reach into another session's browser memory; only a reload can.

The handler is **POST-only** so that no third-party page can log a member out with an `<img>` tag. Two tests hold this: `signout-clears-cache.test.ts` asserts the 303 and the absence of a GET handler, and `member-actions.test.tsx` asserts the button posts to the route rather than calling an action — a refactor back to an action would look like a tidy-up and would silently restore the leak.

Verified on production: after signing out, the JavaScript context is destroyed, `/swipe` redirects to `/login`, the back button cannot restore a signed-in page, and a second member signing in on the same tab sees only their own profile.

---

3. Authorisation

Four layers. Each assumes the ones above it may fail.

Layer 1 — the edge proxy (`proxy.ts`)

Runs before any page renders. Refreshes the session and redirects unauthenticated requests away from `/swipe`, `/matches`, `/listing`, `/profile`, `/chat` and `/browse/[id]/chat`.

Deliberately **page routes only**: an API caller must receive a JSON 401, not an HTML redirect to a login page. API routes check authorisation inside the handler.

Layer 2 — the page

Every signed-in page calls a gate (`requireUser()` / `requireCachedProfile()`) that redirects to `/login` if there is no session and to `/login?error=suspended` if the account is suspended.

Layer 3 — the Server Action

Every mutation begins with `requireUser()` — **uncached, on every write.** See §8 for why this distinction is drawn deliberately.

Layer 4 — Row Level Security

The one that actually matters. 72 policies across 21 tables. Examples of the shapes used:

RULE	HOW IT IS EXPRESSED
A member may only edit their own profile	`using (user_id = auth.uid())`
Private details are owner-only	Separate `profile_details` table, owner-only policy, plus a `public_profile_details()` function returning only fields the owner chose to expose
Only participants may read a conversation's messages	Policy joins through `conversations` and `listing_residents`
A blocked member disappears both ways	`blocked_user_ids()` helper used inside policies
A room's household may edit its listing	`can_manage_listing()` — owner **or** confirmed resident
Chats survive a deleted room	A second `listings` SELECT policy via `linked_to_listing()`, so history stays readable to people who talked about it

The "no write policy at all" pattern

Three tables have **no insert/update/delete policy whatsoever**:

TABLE	WRITTEN ONLY BY
`suspensions`	`apply_report_suspension()`
`listing_invites`	`invite_listing_roommates()`, `respond_to_listing_invite()`
`app_config`	SECURITY DEFINER functions only

This is the strongest pattern in the schema and worth being able to explain. When a table must only ever be written one specific way, **give it no write policy and one `SECURITY DEFINER` function**. There is then no path to a wrong write — as opposed to a policy that tries to enumerate every wrong write and might miss one.

The concrete consequence: **a member cannot lift their own suspension.** Not because a policy forbids it, but because no route to that write exists.

Ownership that cannot be reassigned

A trigger (`listings_owner_is_permanent`) refuses any plain `UPDATE` of `listings.owner_id`. The single legitimate case — handing a shared listing to a roommate when an account is deleted — happens inside `delete_own_account()`, which sets a transaction-local GUC that nothing else in the schema sets. So the listing survives its creator, and there is still no way to steal one.

---

4. What requires a session

PUBLIC	REQUIRES A SESSION
`/browse` — the room list	Swipe deck
A room's page and photos	Chat, and starting a conversation
The map	Creating or editing a listing
Signup / login / password reset	Profile, and viewing another member's profile
	Hearts, viewing history, scheduling a viewing
	Settings, reporting, blocking
	Anything in `/api/*` that touches member data

Browsing is intentionally open — a marketplace that demands signup before showing supply cannot build the liquidity it needs. Everything that reads or writes a *person* requires a session.

---

5. Preventing access to another member's data

Beyond RLS, three specific mechanisms:

Per-member cache keys. Every private cache entry is keyed and tagged with the member's id — `deck:<userId>`, `profile:<userId>`, `saved:<userId>`, `chat:<userId>`. `cache-invalidation.test.ts` asserts that every per-member tag carries the acting member's id: a tag that forgot it would be **one shared cache key for the whole application**, which is exactly the shape of a cross-user leak.

Private caches never reach a shared store. `"use cache: private"` results live in the requesting browser's memory only. `"use cache"` (shared) is used for exactly one thing — the public room list — and it goes through a deliberately **cookie-free** Supabase client (`lib/supabase/public.ts`, `server-only`) so its output cannot become member-specific.

Field-level exposure control. `public_profile_details()` returns only the fields a member chose to make visible. Contact details default to hidden. The split between `profiles` (readable by members) and `profile_details` (owner-only) means a newly added private column is private *by default* rather than by remembering.

Storage. Chat photos live in a private bucket, served as short-lived signed URLs; a photo path must sit under its own conversation's prefix, checked server-side.

---

6. Input validation

Three layers, described in full in the Technical Design:

1. **Client** — convenience only, assumed bypassable.
2. **Server** — six Zod schemas. The Server Action parses before touching the database, and the TypeScript types are inferred from the schemas so the two cannot drift.
3. **Database** — `CHECK` constraints, enums, foreign keys, unique indexes.

Security-relevant specifics:

- **SQL injection is structurally absent.** All access goes through PostgREST or parameterised RPC. No string-built SQL exists anywhere in the codebase.
- **XSS.** React escapes by default; there is no `dangerouslySetInnerHTML` in the product.
- **Open redirects.** `sanitizeNextPath()` guards every `?next=` parameter, so an emailed `?next=https://evil.example` cannot bounce a member off-site after login. Covered by `redirect.test.ts`.
- **Uploads.** Type and size checked on both sides; a chat image path must belong to its conversation.
- **Photo content.** An image is stored **only if** an AI check agrees it shows what its slot claims — a bedroom slot cannot hold a picture of a car.
- **URL parameters** are parsed with `.catch()` fallbacks, so hostile query strings produce defaults rather than errors or unfiltered results.

---

7. Protecting API calls and secrets

API routes

Route Handlers authenticate inside the handler and return JSON 401 rather than redirecting. Every query underneath still runs under RLS, so an authenticated caller poking at another member's id gets an empty result rather than data.

`/api/places` exists specifically **so the browser never calls Overpass directly** — the upstream call, and any future key or quota on it, stays server-side.

Secrets

SECRET	WHERE IT LIVES	REACHES THE BROWSER?
`NEXT_PUBLIC_SUPABASE_URL`	Env + build arg	Yes — designed to be public
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Env + build arg	Yes — public by design, powerless without a session, governed by RLS
`SUPABASE_SERVICE_ROLE_KEY`	Vercel **sensitive** production env var	**Never**
`GEMINI_API_KEY`	Vercel sensitive env var	**Never**
`SMTP_*` credentials	Vercel sensitive env vars	**Never**
`GOOGLE_CLIENT_SECRET`	Vercel sensitive env var	**Never**
`SUPABASE_ACCESS_TOKEN`	Local only, for the auth-config script	**Never**

Rules that are followed rather than assumed:

- **`.env*.local` is gitignored**, and no environment file is tracked. `.env.example` documents the variable *names* with no values.
- **The service-role key is never passed as a build argument.** Build args are baked into the build; runtime secrets are runtime env vars. This distinction is written into the deployment notes because getting it wrong is silent.
- **The anon key being public is not a weakness.** It identifies the project; it grants nothing. Every table it can reach has RLS enabled.
- **A Stop hook auto-commits after every working session**, which makes "never put a secret in a tracked file" a standing rule rather than a habit.

---

8. A deliberate trade-off, disclosed

The identity read (`auth.getUser()` plus the suspension lookup) is **cached per browser for rendering**, with a 300-second window.

Why: the App Shell prerender advances through cached reads and stops at the first uncached one. That single round-trip sat at the top of every page and kept everything behind it out of the prefetch, costing ~300 ms of loading skeleton on every navigation between tabs.

What it costs: a suspension applied mid-session now takes effect within the cache window rather than on the very next page load. The check is **not removed** — `suspensions` is still read and both gates still redirect on it — it simply refreshes at most once per window. This was raised with the product owner and accepted explicitly.

What it does not cost, by design:

- The **edge proxy still calls `auth.getUser()` uncached on every request** to a protected route. That is the real gate, and it is untouched. An expired or revoked session cannot reach these pages at all.
- **Every Server Action still uses the uncached `requireUser()`.** A cached identity is never what authorises a write. `cached-session-boundary.test.ts` fails if any action in `app/actions/` calls a cached reader — both spellings compile and both "work", so only a test can hold that line.
- **RLS is entirely unaffected.** Every query still runs on the cookie-bearing client, so Postgres decides what comes back regardless of what the cache believes.

Disclosing this is the point. A security document that lists only strengths is not a security document.

---

9. Remaining risks

Ordered by how much they would actually matter.

1. **No identity verification.** Email confirmation only. Anyone can create an account with a working address, and members meet in person. This is the largest real-world risk in the product and is not solvable with code alone — it needs ID verification, and a policy for what happens when it fails.
2. **Suspension latency** — up to 5 minutes, per §8.
3. **No rate limiting on most writes.** Only auth email is throttled. Message sending, listing creation and report submission are protected by RLS but not by volume limits, so a determined authenticated member could spam.
4. **No CSRF token on the sign-out form.** Mitigated by POST-only and `SameSite=Lax` session cookies, which do not travel on a cross-site POST. Low severity — the worst outcome is being logged out — but it is a gap.
5. **Photo moderation is single-model.** The Gemini check is a content-*type* check, not a safety check. It confirms a bedroom photo shows a bedroom; it is not a general abuse filter.
6. **No audit log.** There is no record of who read what. A breach could not be scoped after the fact.
7. **Message content is not encrypted at rest** beyond Supabase's disk encryption. Anyone with database access can read chats.
8. **No 2FA.**
9. **Reports are not actioned automatically.** `apply_report_suspension()` exists; nothing calls it on a threshold. A moderator would have to act, and there is no moderator.
10. **Dependency risk.** No automated vulnerability scanning in CI.

---

10. What a hardened version would add

In priority order:

1. **Rate limiting** on writes — per member and per IP, at the edge.
2. **A real moderation queue** — reports surfaced to a human, with `apply_report_suspension()` wired to a threshold.
3. **2FA**, at least optional, for accounts sharing contact details.
4. **An audit log** for reads of sensitive fields, so a breach can be scoped.
5. **CSRF tokens** on state-changing form posts, rather than relying on SameSite alone.
6. **Automated dependency scanning** — Dependabot plus `npm audit` in CI.
7. **A Content Security Policy** — currently absent; would harden against injected script even though React escapes by default.
8. **Shorter session lifetimes** with silent refresh.
9. **A tested backup and restore procedure.** Supabase takes backups; nobody has restored one, and an untested backup is a hope rather than a plan.
10. **RLS policy tests** that run real queries as two different members against a test project — the one layer the unit suite cannot reach today.
